

1. Dati del progetto

Nome e cognome: Mohamed El Halawti

Classe: 3B.TLC

Titolo del gioco: No Sleep Run: Cobweb Edition

Data della consegna: 1/06/2026

2. Idea del gioco

Che tipo di gioco è?

"No Sleep Run" è un videogioco platform endless-runner (a scorrimento infinito) in 2D.

Qual è l'obiettivo?

L'obiettivo del giocatore è sopravvivere il più a lungo possibile fuggendo da un fantasma/incubo che lo insegue. Bisogna saltare tra i tetti dei palazzi, evitare ostacoli (come ragnatele, droni, pipistrelli e casse) e raccogliere monete per aumentare il punteggio. Se si cade nel vuoto o se il fantasma raggiunge il giocatore, la partita finisce.

Cosa lo rende originale?

Rispetto ai classici tutorial online, questo gioco ha due particolarità tecniche:

L'audio procedurale: Nessun file .wav o .mp3 esterno è stato utilizzato. Tutti i suoni (salti, sciolate, musica ambientale, jumpscare) sono generati matematicamente in tempo reale usando le onde sinusoidali tramite le librerie math e struct.

Sistema di parkour dinamico: Il personaggio ha animazioni basate su primitive geometriche (non su sprite prefatti) che variano in base all'azione, incluse capriole randomiche durante il doppio salto e una sciolata divisa in tre fasi fluide.

3. Analisi e progettazione

3.1 Regole del gioco

Movimento: Corsa automatica. Il giocatore preme SPAZIO per saltare (o doppio salto) e FRECCIA GIÙ per scivolare.

Ostacoli: Le casse fermano il giocatore (permettendo al fantasma di avvicinarsi); le ragnatele rallentano drasticamente la velocità.

Power-up: Ci sono potenziamenti che conferiscono vantaggi temporanei: Hyper Dash (velocità), Ponte Spaziale (pavimento di sicurezza contro le cadute), Mega Salto ed Esorcismo (allontana il fantasma).

Punteggio: Aumenta nel tempo in base alla velocità percorsa, con un bonus di 100 punti per ogni moneta raccolta.

3.2 Struttura del programma

Il gioco è sviluppato interamente in Python. Le librerie principali usate sono:

pygame: Per il rendering grafico, la gestione della finestra e degli input.

math e struct: Per i calcoli della fisica e la sintesi dei suoni.

random: Per la generazione procedurale del mondo.

Ciclo di gioco (Pseudocodice):

All'avvio: Carica High Score e mostra il Menu Iniziale

Se premo Spazio: Passa alla schermata Recap (Guida ai comandi)

Se premo Spazio di nuovo: Inizializza giocatore, fantasma e piattaforme.

Finché il gioco è in stato "PLAYING":

Input: Legge Spazio (salto) o Giù (scivolata).

Aggiornamento: > * Calcola la fisica (gravità, collisioni del Player).

Muove le piattaforme e lo sfondo verso sinistra.

Genera nuove piattaforme quando le vecchie escono dallo schermo.

Condizioni di sconfitta: Se il Player cade ($Y > \text{altezza_schermo}$) o se il rettangolo del Fantasma tocca il Player -> Cambio stato in "GAMEOVER".

Disegno: Pulisce lo schermo, disegna sfondo, piattaforme, ostacoli, Player e UI.

3.3 Strutture dati importanti

Classe Player: Gestisce il protagonista. Contiene variabili di stato (es. `is_jumping`, `is_sliding`, `is_dead_falling`), coordinate (`x`, `y`), timer per le animazioni e i metodi `update()` per la fisica e `draw()` per renderizzare i poligoni che formano il corpo.

Classe Ghost: Gestisce il nemico che insegue il giocatore, con una logica di levitazione fluttuante (basata sul seno) e l'emissione di particelle.

Lista platforms: È una lista di dizionari (Dictionaries). Ogni dizionario rappresenta un tetto e contiene dati come `x`, `y`, `larghezza`, oltre a liste nidificate per le `windows`, le monete (`coins`) e gli `obstacles`. Questo ci permette di generare il mondo all'infinito rimuovendo le piattaforme vecchie e aggiungendone di nuove.

4. Sviluppo e uso dell'AI

4.1 Come hai lavorato

Ho iniziato dall'idea base di un omino che corre. Poi mi sono concentrato prima sulla fisica del salto, poi sulla generazione infinita delle piattaforme.

4.2 Come hai usato l'AI

Per questo progetto ho utilizzato l'assistente AI (Gemini) in modo attivo, soprattutto per risolvere problemi tecnici specifici e per funzioni avanzate che esulavano dalle basi di Pygame.

Esempi concreti di utilizzo:

Sintesi Audio: Ho chiesto all'AI come generare suoni senza usare file esterni. L'AI mi ha spiegato come riempire un buffer di byte usando `math.sin` e passarlo a `pygame.mixer.Sound`, spiegandomi concetti matematici legati alle frequenze.

Risoluzione Bug: [Qui puoi usare esattamente quello che è successo poco fa!] Ho avuto un crash con l'errore `TypeError: this function got an`

unexpected keyword argument 'border_radius'. Ho fornito all'AI il traceback dell'errore. L'AI mi ha spiegato che la funzione `pygame.draw.polygon` non accetta il parametro `border_radius` (che invece funziona su `rect`), dicendomi esattamente quale riga correggere.

Rifinitura Animazioni: Ho chiesto idee su come animare la corsa, e ho riadattato il codice fornito per creare l'animazione personalizzata della scivolata parkour e della super velocità.

In tutti i casi, non ho fatto un semplice "copia-incolla" cieco, ma ho dovuto integrare il codice suggerito all'interno delle mie Classi (Player e Ghost), adattando le variabili (`bx`, `by`, `self.y`) alla mia logica.

5.2 Bug e problemi

A volte le casce vengono generate troppo vicine alle ragnatele rendendo il salto molto difficile
Avrei voluto inserire un menu di pausa ma non ho fatto in tempo

6. Conclusioni e riflessione personale

Mi sono molto divertito creando questo gioco, l'uso della mia fantasia è stato molto bello ed unico, e soprattutto la costante voglia di migliorarlo e renderlo sempre più figo